КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

ІМЕНІ ТАРАСА ШЕВЧЕНКА

Факультет інформаційних технологій

Кафедра кібербезпеки та захисту інформації

Лабораторна робота №4

Тема: Авторизація доступу до ресурсів і захист від IDOR/BOLA

з дисципліни:

«ПРИКЛАДНІ ТЕХНОЛОГІЇ ПРОГРАМУВАННЯ В ІНФОРМАЦІЙНІЙ БЕЗПЕЦІ»

Виконала: студентка групи КБ-43 Котенок Вікторія

Перевірив: Бабенко Юрій Михайлович

Київ 2026

МЕТА РОБОТИ

Відокремити авторизацію від автентифікації: після перевіреного входу з ЛР 3 сервер має сам вирішувати для кожного запиту «суб'єкт + дія + ресурс + контекст», а не довіряти самому лише факту наявності cookie чи переданому клієнтом identifier. Відтворити на контрольованому локальному стані горизонтальне підвищення привілеїв (IDOR/BOLA), усунути його обмеженням запиту за суб'єктом і відокремленим resource-based рішенням, узгодити політику відповідей 404 для чужого й неіснуючого ресурсу, закрити вертикальне підвищення привілеїв рольовою перевіркою з контекстом стану, не дозволити DTO підмінити серверні поля, додати безпечний аудит відмов і підтвердити поведінку автоматизованими сценаріями A-01…A-12, зафіксувавши перевірений стан тегом v0.4.0.

1. КОНТЕКСТ І МЕЖА ВИКОНАННЯ

Рівень: відмінний («Відмінно»). Рівні накопичувальні, тож виконано достатній (матриця головної action, IDOR/BOLA PoC, owner з principal, A-02, A-03, A-03R), добрий (централізоване рішення, list-scoping, update/delete, triage, child, audit, політика 404, A-01, A-04–A-10, A-12) і відмінний (відокремлений resource-based handler, A-11 для search і severity-summary, окремий finding додаткової проблеми, обґрунтування видимості й least privilege).

Baseline і base tag: власний кумулятивний проєкт, baseline 2-A «Трекер інцидентів»; base tag v0.3.0, hash 95c7d9e8678e85d54da49fa1a53c45daea840180.

Головна action і політика: головна protected action — details/read (GET /api/incidents/{id}); для чужого й неіснуючого ресурсу застосовано політику 404 (однакова відповідь, щоб не розкривати існування ресурсу). Маршрути рівня: GET /api/incidents/{id}, GET /api/incidents (list), update і delete для Incident, POST /api/incidents/{id}/triage, GET /api/incident-comments/{id}, GET /api/incidents/search, GET /api/incidents/severity-summary.

ЛР 3 після тегу: не виправлялася; тег v0.3.0 не пересувався. Гілку ЛР 4 створено від нього.

Таблиця 1 — Відтворення

Параметр	Значення
Рівень / оцінка	відмінний / «Відмінно»
Автор / група	Котенок Вікторія, КБ-43
Baseline	2-А «Трекер інцидентів»
Base tag v0.3.0	95c7d9e8678e85d54da49fa1a53c45daea840180
Гілка	lab/4-authorization (від v0.3.0)
Головна action / політика відмов	details/read (GET /api/incidents/{id}) / 404
Vulnerable commit	8396023d73308eddb4b31acef2993853514925a4
Fixed commit (перевірений) / tag	e56926850bb1913db1f6cad4085efd47093f954d / v0.4.0
ОС / SDK	Windows / 10.0.303
Фактичний результат тестів	всього 40 тестів, збоїв 0 (до змін ЛР 4 — 24 тести, збоїв 0)
Remote	https://github.com/vkotenok35-hub/securelab-kotenok

Відтворення стартового стану

Робоче дерево чисте, гілку створено від власного тегу v0.3.0; останній commit гілки містить tag: v0.3.0. Пароль навчальних користувачів задано лише змінною поточного термінала SeedUsers__Password, без запису у файли, звіт чи Git; helper login-and-read.py --check підтверджує лише факт наявності змінної та придатність JSON, не показуючи пароля.

```
git status
git switch -c lab/4-authorization v0.3.0
git log --oneline --decorate -1
```

```
PS D:\laba1> git status
On branch lab/4-authorization
nothing to commit, working tree clean
PS D:\laba1> git log --oneline --decorate -1
95c7d9e (HEAD -> lab/4-authorization, tag: v0.3.0, origin/lab/3-authentication, lab/3-authentication) Implement cooki
e authentication and protected incident creation
```

Рисунок 1 — Гілка lab/4-authorization від тегу v0.3.0

Далі піднято PostgreSQL, виконано reset і регресійні тести попередніх робіт; reset друкує «Локальні навчальні дані очищено та повторно заповнено seed-значеннями.» і завершується. Після reset підтверджено login за Alice, Bob

і Morgan та їхні ролі (Alice, Bob — Reporter; Morgan — Analyst; Admin — Administrator). Стартовий стан перевірено повним прогоном: 24 тести, збоїв 0.

```
docker compose --env-file infra/.env.example -f infra/compose.yaml up -d --wait
dotnet run --project src/SecureLab.Api -- --reset-database
dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj --configuration
Release
```

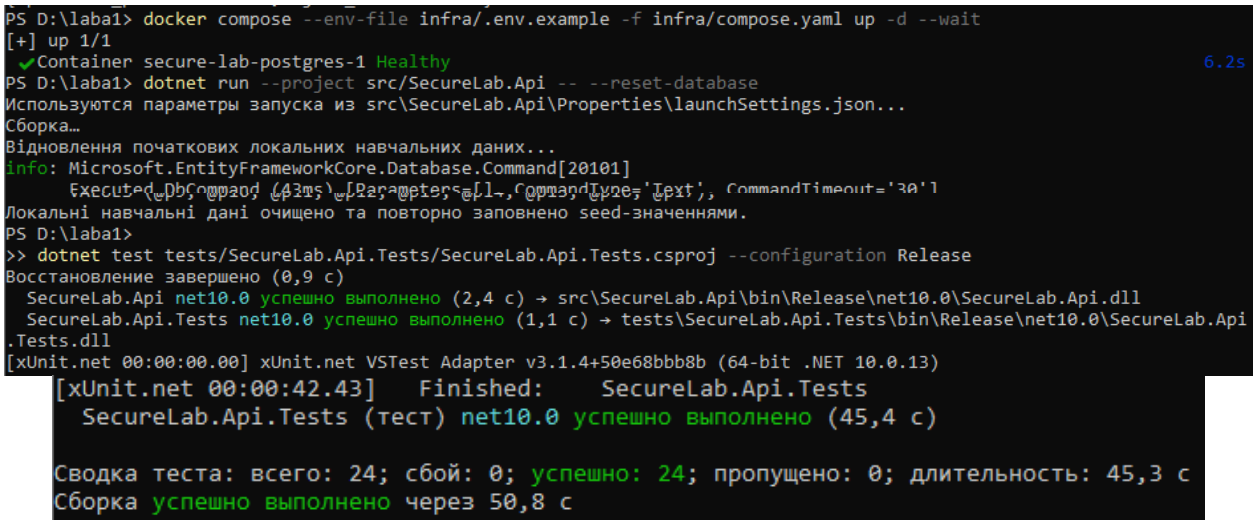


Рисунок 2 — Відтворення стану: reset і тести перед роботою

2. МАТРИЦЯ І SERVER-SIDE РІШЕННЯ

Матрицю доступу заповнено у файлі docs/access-matrix.md за ролями Reporter, Analyst, Administrator і правилом володіння ресурсом. Кожну клітинку рядків рівня замінено на allow, deny або умову словами, а для deny вказано HTTP-статус. Права випливають із предметного правила baseline 2-A, а не з назви ролі: Administrator не отримує автоматичного обходу ownership, Analyst має читання й лише перехід New → Triaged.

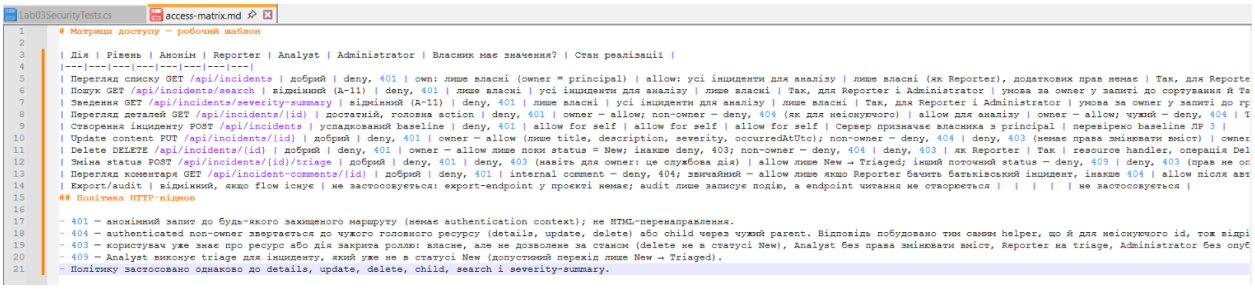


Рисунок 3 — Матриця контролю доступу

Таблиця 2 — Правила доступу (стислий витяг матриці)

Дія	Reporter	Analyst	Administrator	Правило і deny-статус
Деталі GET /{id}	owner — allow; інший — deny, 404	allow (для аналізу)	лише власні (як Reporter)	resource handler: owner з principal; чужий і неіснуючий — 404
Список GET /api/incidents	лише власні	усі для аналізу	лише власні	query scoping за owner; чужі не повертаються
Update / Delete	owner — allow (delete лише в статусі New); інший — deny, 404	deny, 403	як Reporter	resource handler; стан без змін; owner не в New на delete — 403
Triage New→Triaged	deny, 403	allow лише New → Triaged; інший status — 409	deny, 403	роль перевіряється в endpoint; сервер задає next status; audit authorization.denied
Internal comment / child	internal — 404; звичайний — лише через власний parent	allow після авторизації parent, включно з internal	як Reporter	child через parent; internal не в проєкції Reporter

Дія	Reporter	Analyst	Administrator	Правило і deny-статус
Search / severity-summary	лише власні	усі для аналізу	лише власні	умова в запиті до сортування й групування (A-11)

Де реалізовано рішення

Query scoping. Для list, search і severity-summary умова за owner і роллю з HttpContext.User стоїть у самому EF-запиті (метод VisibleTo): для list — до проєкції, для search — до сортування й Take(50), для severity-summary — до групування за severity. Тому заборонений рядок взагалі не завантажується в пам’ять. Для details ресурс спершу завантажується за id, а рішення про доступ ухвалює resource-based handler (нижче) до проєкції відповіді.

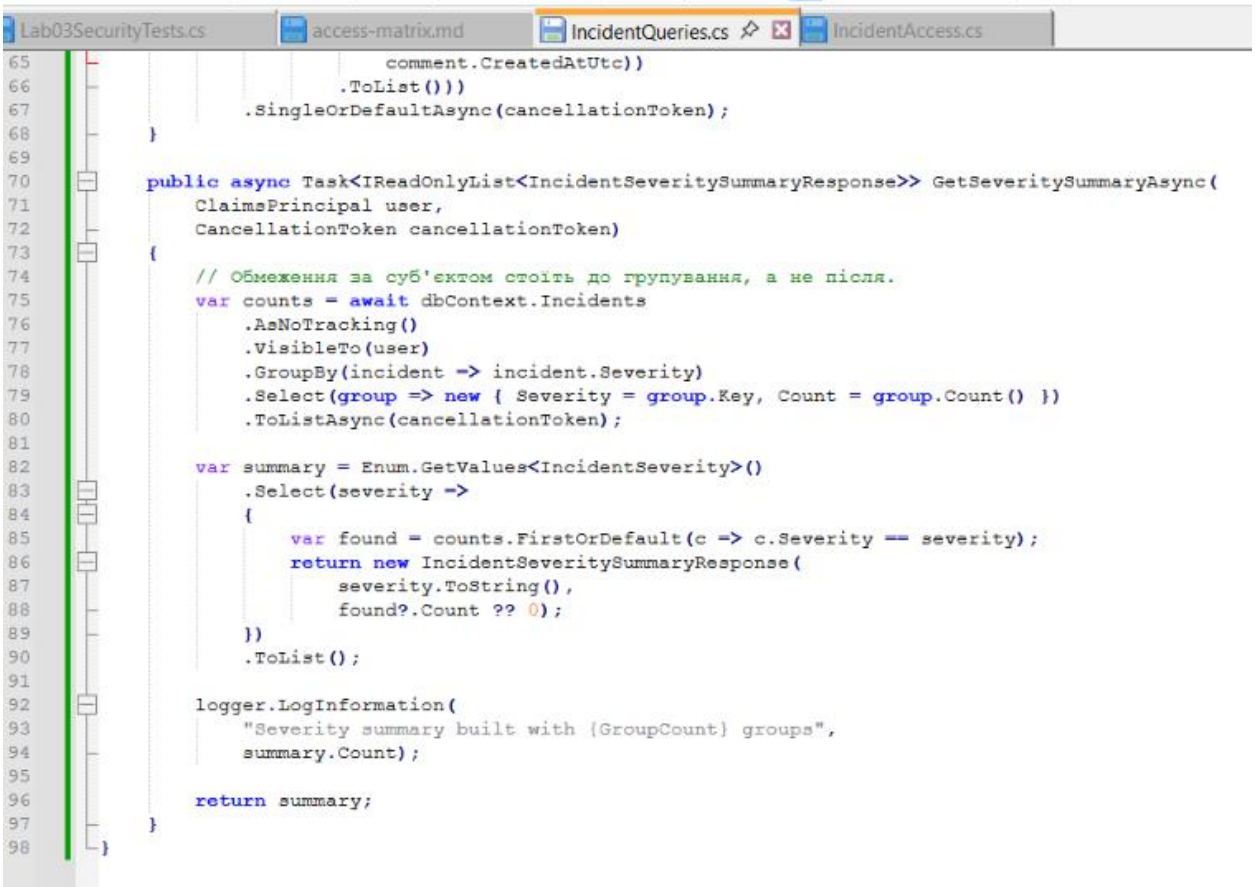


Рисунок 4 — Query scoping за суб’єктом

Відокремлене resource-based рішення. Для details, update, delete і child застосовано окремий resource-based handler (IAuthorizationHandler), який ухвалює рішення за трійкою principal + resource + operation після завантаження ресурсу; handler зареєстровано в Program.cs. Поточний користувач береться лише з principal, а не з body, URL, query чи заголовка клієнта.

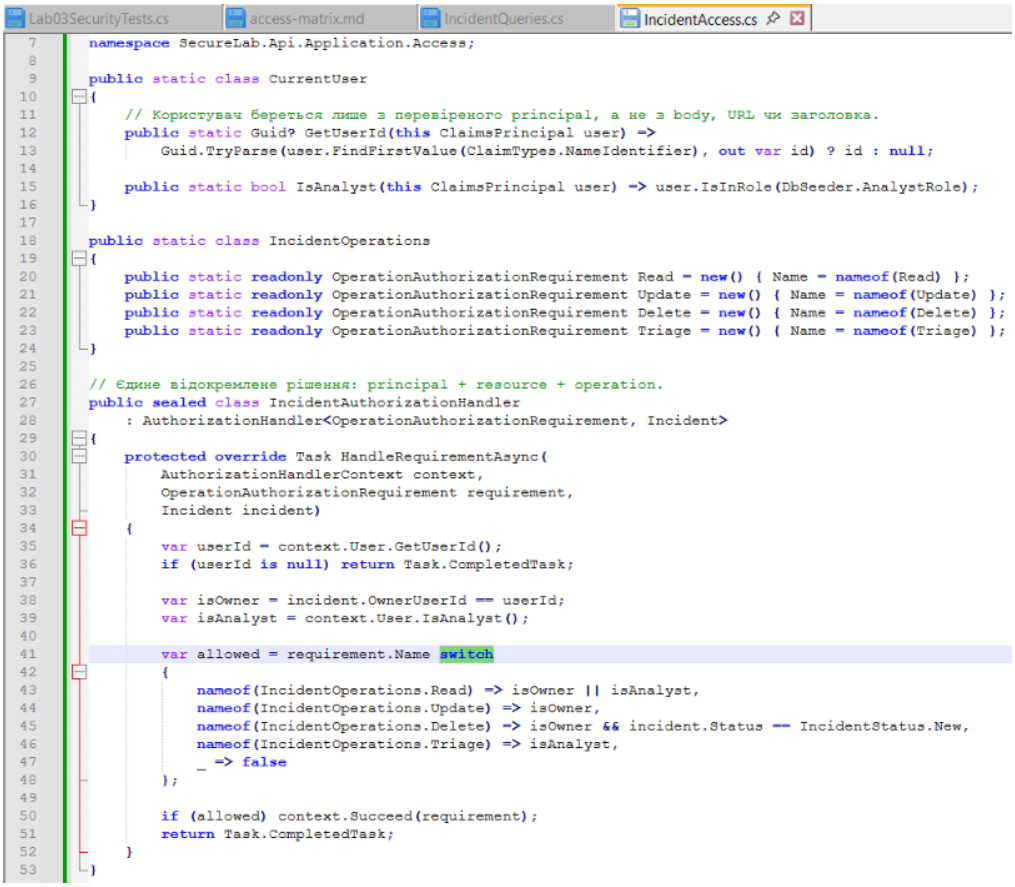


Рисунок 5 — Відокремлене resource-based рішення

Server-managed поля. Request DTO операцій містить лише дозволені поля; OwnerUserId, роль і захищений status встановлює сервер. Зайве поле (owner, role, status) у тілі ігнорується контрактом (A-08a) або відхиляється (A-08b) і не змінює захищеного стану. Для triage сервер сам визначає наступний статус після перевірки поточного.

Lab02Endpoints.cs | IncidentRequests.cs | IncidentEndpointTests.cs | SearchMechanicsTests.cs

```
1   namespace SecureLab.Api.Presentation.Contracts;
2
3   // Лише content-поля. OwnerUserId, role, status, id і службові часи визначає сервер.
4   public sealed record UpdateIncidentRequest(
5       string? Title, string? Description, string? Severity, DateTimeOffset? OccurredAtUtc);
```

Lab02Endpoints.cs | IncidentMutationEndpoints.cs | IncidentRequests.cs | IncidentEndpointTests.cs | SearchMechanic

```
54
55          // Рішення передує будь-якій зміні стану.
56          if (!(await authorization.AuthorizeAsync(user, incident, IncidentOperations.Update)).Succeeded)
57              return await DenyAsync(authorization, user, incident);
58
59          var now = DateTimeOffset.UtcNow;
60          var errors = new Dictionary<string, string[]>();
61
62          if (request.Title is { Length: > 160 })
63              errors["title"] = ["Заголовок не може перевищувати 160 символів."];
64          if (request.Description is { Length: > 4000 })
65              errors["description"] = ["Опис не може перевищувати 4000 символів."];
66
67          var title = request.Title?.Trim() ?? "";
68          var description = request.Description?.Trim() ?? "";
69          if (title.Length == 0 && !errors.ContainsKey("title"))
70              errors["title"] = ["Заголовок обов'язковий."];
71          if (description.Length == 0 && !errors.ContainsKey("description"))
72              errors["description"] = ["Опис обов'язковий."];
73          if (title.Length > 0 && !title.Any(char.IsLetter) && !errors.ContainsKey("title"))
74              errors["title"] = ["Заголовок має містити хоча б одну літеру."];
75
76          IncidentSeverity severity = default;
77          var severityValid = Enum.TryParse<IncidentSeverity>(request.Severity, true, out severity)
78                              && Enum.IsDefined(severity);
79          if (!severityValid)
80              errors["severity"] = ["Допустимі значення: Low, Medium, High, Critical."];
81
82          var occurred = request.OccurredAtUtc;
83          if (occurred is null)
84              errors["occurredAtUtc"] = ["Дата й час події обов'язкові."];
85          else if (occurred.Value > now.AddMinutes(5))
86              errors["occurredAtUtc"] = ["Подія не може бути в майбутньому."];
87
88          if (severityValid
89              && (severity == IncidentSeverity.High || severity == IncidentSeverity.Critical)
90              && description.Length > 0 && description.Length < 40
91              && !errors.ContainsKey("description"))
92              errors["description"] = ["Для High і Critical опис має містити щонайменше 40 символів."];
93
94          if (errors.Count > 0) return Results.ValidationProblem(errors);
95
96          if (await db.Incidents.AnyAsync(
97                  item => item.Id != id && item.Title == title && item.Status != IncidentStatus.Closed, ct))
98              return Results.Problem(title: "Інцидент уже існує",
99                  detail: "Незакритий інцидент із таким заголовком уже зареєстровано.",
100                 statusCode: StatusCodes.Status409Conflict);
```

Рисунок 6 — Server-managed поля та межа DTO

3. ВРАЗЛИВИЙ СТАН І POC

До виправлення головна action details читала інцидент лише за id (IncidentQueries.GetDetailsAsync із .Where(incident => incident.Id == id)) і не порівнювала його власника з поточним користувачем. Маршрут навіть не вимагав автентифікації: анонімний запит curl.exe до details Bob повертав 200. Це контрольований локальний vulnerable стан на синтетичних даних.

PoC мінімальний: той самий cookie Alice, змінено лише id на ресурс Bob. Жодного перебору ідентифікаторів, сканерів чи небезпечних payload. Сценарій зафіксовано у tests/http/lab-04-access.http (login Alice, Bob, Morgan і read-запити); cookie й пароль у файл не записано, пароль береться зі змінної середовища.

```
py -3 ../lr04-resources/commands/login-and-read.py --user alice --incident alice
py -3 ../lr04-resources/commands/login-and-read.py --user alice --incident bob
```

До виправлення запит Alice до власного ресурсу повертає 200 і id …0001, а запит Alice до ресурсу Bob повертає 200 і дані Bob (ownerDisplayName «Боб Мельник») — це і є IDOR/BOLA.

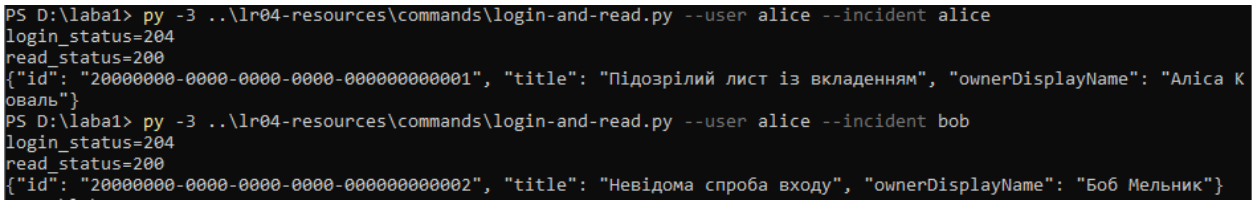


Рисунок 7 — IDOR/BOLA до виправлення: Alice читає ресурс Bob

Стан «до» збережено окремим vulnerable commit на гілці lab/4-authorization. Він лишається лише в приватному навчальному репозиторії, не публікується у відкритому remote і не позначається фінальним тегом. Vulnerable commit: 8396023d73308eddb4b31acef2993853514925a4.

```
git add tests/http/lab-04-access.http docs/
git commit -m "Reproduce IDOR on incident details"
git rev-parse HEAD
```

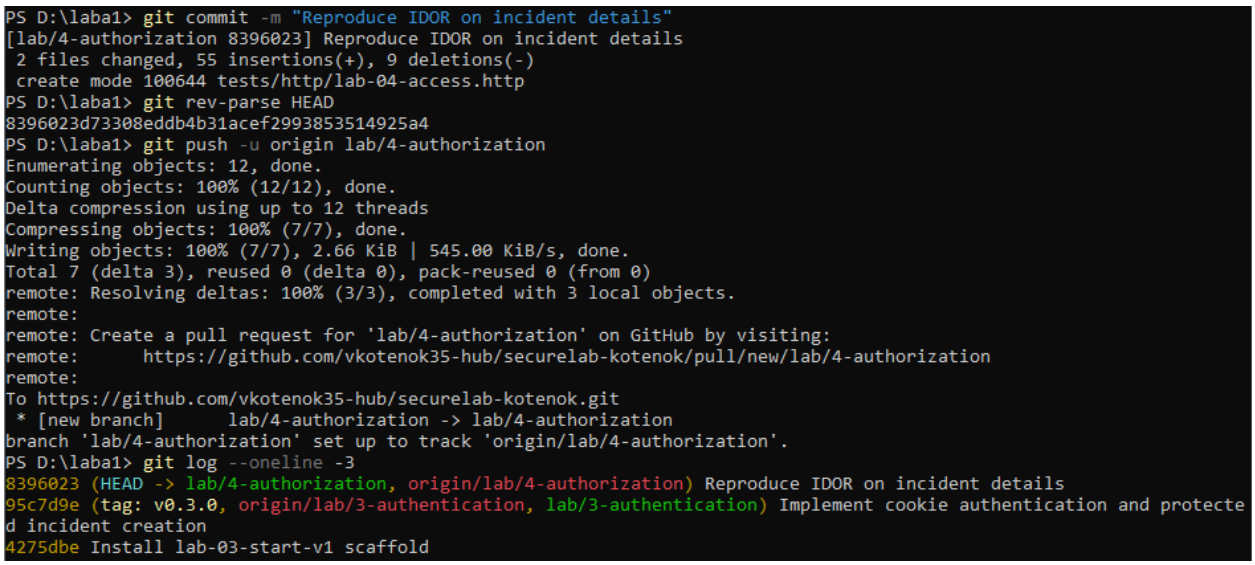


Рисунок 8 — Vulnerable commit

Першопричина. Пошук лише за id → повернення ресурсу → відсутність порівняння з principal і ресурсного рішення. «Id у URL» описує механізм запиту, а не дефект: URL-id нормальний, доки сервер перевіряє право на ресурс. Відсутнє саме обмеження запиту за owner і виклик централізованого рішення.

Контрольна точка CP-01. Показано заповнений рядок матриці головної action, два запити (Alice → Alice, Alice → Bob), пояснення, чому однакова роль Reporter не робить ресурс Bob доступним Alice (право випливає з ownership, а не з ролі), і місце майбутнього server-side рішення.

4. ВИПРАВЛЕННЯ ТА RETEST

Після vulnerable commit правило з матриці реалізовано на сервері: list, search і severity-summary звужено query scoping за owner і роллю, для окремого ресурсу (details, update, delete, child) застосовано resource-based handler, а 404 узгоджено для чужого й неіснуючого ресурсу. Перевірка передує відповіді та будь-якій зміні стану. Fixed стан збережено окремим commit e56926850bb1913db1f6cad4085efd47093f954d, на який поставлено анотований тег v0.4.0.

```
git add <змінені файли коду й тестів>
git commit -m "Authorize incident access by owner and role"
git rev-parse HEAD
```

Повторено ті самі запити Alice → Alice і Alice → Bob. Власний ресурс лишається доступним (позитивна регресія), а чужий повертає 404 Problem Details «Інцидент не знайдено», ідентичний відповіді для неіснуючого id; даних Bob у відповіді немає.

```
PS D:\laba1> py -3 ..\lr04-resources\commands\login-and-read.py --user alice --incident alice
login_status=204
read_status=200
{"id": "20000000-0000-0000-0000-000000000001", "title": "Підозрілий лист із вкладенням", "ownerDisplayName": "Алиса К
оваль"}
```

Рисунок 9 — Позитивна регресія: Alice читає власний ресурс

```
PS D:\laba1> py -3 ..\lr04-resources\commands\login-and-read.py --user alice --incident bob
login_status=204
read_status=404
{"title": "Інцидент не знайдено", "status": 404, "traceId": "00-0386a6a028c33d3da819bff77ac1bf7f-15bd5b9815de874c-00"
}
```

Рисунок 10 — Alice → Bob після виправлення

A-10: два однакові за структурою запити (чужий і неіснуючий id) дають однакові status, Content-Type і тіло після вилучення traceId. A-03R: Morgan у ролі Analyst читає деталі інциденту Alice (200) для аналізу, і це не надає права довільної mutation.

```
98      // A-10: чужий і неіснуючий id нерозрізнювані (однакові status, Content-Type і тіло без traceId)
99      [Fact]
100     public async Task A10_ForeignAndUnknownId_ReturnIdenticalNotFound()
101     {
102         using var client = await LoginAs("alice");
103         using var foreign = await client.GetAsync($"/api/incidents/{DbSeeder.BobIncidentId}");
104         using var unknown = await client.GetAsync($"/api/incidents/{Guid.NewGuid()}");
105
106         Assert.Equal(HttpStatusCode.NotFound, foreign.StatusCode);
107         Assert.Equal(unknown.StatusCode, foreign.StatusCode);
108         Assert.Equal(unknown.Content.Headers.ContentType?.ToString(),
109             foreign.Content.Headers.ContentType?.ToString());
110
111         var foreignBody = JsonNode.Parse(await foreign.Content.ReadAsStringAsync())!.AsObject();
112         var unknownBody = JsonNode.Parse(await unknown.Content.ReadAsStringAsync())!.AsObject();
113         foreignBody.Remove("traceId"); // єдина дозволена відмінність
114         unknownBody.Remove("traceId");
115         Assert.True(JsonNode.DeepEquals(unknownBody, foreignBody));
116     }
```

```
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release --filter A10_ForeignAndUnkno
wnId_ReturnIdenticalNotFound
Восстановление завершено (0,9 с)
  SecureLab.Api net10.0 успешно выполнено (0,6 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,4 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.13)
[xUnit.net 00:00:04.63]   Finished:    SecureLab.Api.Tests
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (6,1 с)

Сводка теста: всего: 1; сбой: 0; успешно: 1; пропущено: 0; длительность: 6,0 с
Сборка успешно выполнено через 9,0 с
```

Рисунок 11 — A-10: нерозрізнюваність 404

```
PS D:\laba1> py -3 ..\lr04-resources\commands\login-and-read.py --user morgan --incident alice
login_status=204
read_status=200
{"id": "20000000-0000-0000-0000-000000000001", "title": "Підозрілий лист із вкладенням", "ownerDisplayName": "Alica K
оваль"}
```

Рисунок 12 — A-03R: аналітичний read Morgan → Alice

Контрольна точка CP-01 (retest). Alice → власний ресурс — allow; Alice → Bob — 404 без даних Bob; відповідь чужому й неіснуючому ресурсу узгоджена.

5. РОЛЬ, TRANSITION, CHILD I AUDIT (ДОБРИЙ РІВЕНЬ)

List обмежено дозволеною множиною: Alice бачить лише власні інциденти (…0001, …0003, …0005), ресурсу Bob у відповіді немає (A-05). Анонімний запит до захищеного маршруту повертає 401 без HTML-перенаправлення (A-01). Для чужого ресурсу всі змінювальні дії (update, delete) дають deny, а стан ресурсу Bob не змінюється (A-04).

```
85      // A-05: список містить лише дозволену множину
86      [Fact]
87      public async Task A05_List_ContainsOnlyOwnIncidents()
88      {
89          using var client = await LoginAs("alice");
90          var incidents = await client.GetFromJsonAsync<List<IncidentListItemResponse>>("/api/incidents");
91
92          Assert.NotNull(incidents);
93          Assert.Contains(incidents, incident => incident.Id == DbSeeder.AliceIncidentId);
94          Assert.DoesNotContain(incidents, incident => incident.Id == DbSeeder.BobIncidentId);
95          Assert.DoesNotContain(incidents, incident => incident.Id == BobSearchIncidentId);
96      }
```

```
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release --filter A05_List_ContainsOn
lyOwnIncidents
Восстановление завершено (0,9 с)
  SecureLab.Api net10.0 успешно выполнено (0,7 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,4 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (6,1 с)

Сводка теста: всего: 1; сбой: 0; успешно: 1; пропущено: 0; длительность: 6,0 с
Сборка успешно выполнено через 9,0 с
```

Рисунок 13 — A-05: список обмежено дозволеною множиною

Вертикальне підвищення привілеїв. Privileged action — POST /api/incidents/{id}/triage (перехід New → Triaged для Analyst). Звичайний Reporter отримує 403, а статус ресурсу не змінюється (A-06, тест перевіряє знімок стану ресурсу до й після). Triage не приймає тіла запиту, тому підставити role, status чи OwnerUserId немає куди: виконавця й наступний статус визначає сервер. Analyst на інциденті у статусі New виконує дозволений перехід (A-07), а повторний triage того самого інциденту дає 409; роль не дає інших прав.

```
106     // A-06: Reporter не виконує привілейований transition навіть над власним ресурсом
107     [Fact]
108     public async Task A06_Reporter_Triage_Returns403_AndStatusUnchanged()
109     {
110         using var alice = await LoginAs("alice");
111         var before = await SnapshotAsync(DbSeeder.SafeTextIncidentId); // власний, status New
112
113         using var response = await alice.PostAsync($"/api/incidents/{DbSeeder.SafeTextIncidentId}/triage", null);
114
115         Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
116         Assert.Equal(before, await SnapshotAsync(DbSeeder.SafeTextIncidentId));
117     }
118
119     // A-07: Analyst виконує дозволений New → Triaged; повторний transition — 409
120     [Fact]
121     public async Task A07_Analyst_Triage_MovesNewToTriaged_AndRepeatConflicts()
122     {
123         using var alice = await LoginAs("alice");
124         using var morgan = await LoginAs("morgan");
125         var id = await CreateAsync(alice, $"A07-{Guid.NewGuid():N}");
126         try
127         {
128             using var first = await morgan.PostAsync($"/api/incidents/{id}/triage", null);
129             Assert.Equal(HttpStatusCode.OK, first.StatusCode);
130             using var document = JsonDocument.Parse(await first.Content.ReadAsStringAsync());
131             Assert.Equal("Triaged", document.RootElement.GetProperty("status").GetString());
132             Assert.Equal(IncidentStatus.Triaged, (await LoadAsync(id))!.Status);
133
134             using var second = await morgan.PostAsync($"/api/incidents/{id}/triage", null);
135             Assert.Equal(HttpStatusCode.Conflict, second.StatusCode);
136         }
137         finally { await DeleteIncidentAsync(id); }
138     }
```

```
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release --filter "FullyQualifiedName
~A06_|FullyQualifiedName~A07_"
Восстановление завершено (1,0 с)
  SecureLab.Api net10.0 успешно выполнено (0,7 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,5 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.13)
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (7,5 с)

Сводка теста: всего: 4; сбой: 0; успешно: 4; пропущено: 0; длительность: 7,5 с
Сборка успешно выполнено через 10,8 с
```

Рисунок 14 — A-06 і A-07: вертикальне підвищення та дозволений transition

Overposting. Update із підробленими owner, role і status: дозволений content може змінитися, але захищені поля (owner, role, status) лишаються незмінними (A-08a — контракт ігнорує невідоме поле) або запит відхиляється (A-08b).

```
    // A-08a: forged owner/role/status/id у звичайному update не змінюють захищений стан
    [Fact]
    public async Task A08a_UpdateWithForgedProtectedFields_ChangesContentOnly()
    {
        using var alice = await LoginAs("alice");
        var id = await CreateAsync(alice, $"A08-{Guid.NewGuid():N}");
        var newTitle = $"A08-changed-{Guid.NewGuid():N}";
        try
        {
            var body = new
            {
                title = newTitle,
                description = "Оновлений опис без зміни захищених полів.",
                severity = "Medium",
                occurredAtUtc = DateTimeOffset.UtcNow.AddMinutes(-20),
                ownerUserId = DbSeeder.BobId,
                role = "Administrator",
                status = "Closed",
                id = Guid.NewGuid()
            };
            using var response = await alice.PutAsJsonAsync($"/api/incidents/{id}", body)
            Assert.Equal(HttpStatusCode.NoContent, response.StatusCode);

            var saved = await LoadAsync(id);
            Assert.NotNull(saved);
            Assert.Equal(newTitle, saved.Title);
            Assert.Equal(DbSeeder.AliceId, saved.OwnerUserId);
            Assert.Equal(IncidentStatus.New, saved.Status);
        }
        finally { await DeleteIncidentAsync(id); }
    }
```

```
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release --filter "FullyQualifiedName
~A08a_"
Восстановление завершено (1,0 с)
  SecureLab.Api net10.0 успешно выполнено (0,7 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,5 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (6,8 с)

Сводка теста: всего: 1; сбой: 0; успешно: 1; пропущено: 0; длительность: 6,7 с
Сборка успешно выполнено через 10,1 с
```

Рисунок 15 — A-08: overposting не змінює захищені поля

Дочірній ресурс. Reporter за прямим id коментаря Morgan до інциденту Bob отримує 404 (як і для internal comment власного parent), a internal comment не потрапляє у details-проєкцію для Reporter (A-12). Analyst читає internal comment після авторизації parent. Доступ до дочірнього ресурсу перевіряється через право на батьківський інцидент.

```
    // A-12: Reporter не читає чужий і internal коментар; власний звичайний — читає
    [Fact]
    public async Task A12_Reporter_CannotReadForeignOrInternalComment()
    {
        using var alice = await LoginAs("alice");

        using var foreign = await alice.GetAsync($"/api/incident-comments/{DbSeeder.CommentToBobId}");
        using var internalComment = await alice.GetAsync($"/api/incident-comments/{DbSeeder.InternalCommentId}");
        using var own = await alice.GetAsync($"/api/incident-comments/{DbSeeder.CommentToAliceId}");
        var details = await alice.GetStringAsync($"/api/incidents/{DbSeeder.AliceIncidentId}");

        Assert.Equal(HttpStatusCode.NotFound, foreign.StatusCode);
        Assert.Equal(HttpStatusCode.NotFound, internalComment.StatusCode);
        Assert.Equal(HttpStatusCode.OK, own.StatusCode);
        Assert.DoesNotContain("Внутрішня примітка", details, StringComparison.Ordinal);
    }

    // A-12: Analyst читає internal коментар після авторизації parent
    [Fact]
    public async Task A12_Analyst_ReadsInternalComment_ThroughParent()
    {
        using var morgan = await LoginAs("morgan");

        using var internalComment = await morgan.GetAsync($"/api/incident-comments/{DbSeeder.InternalCommentId}");
        var details = await morgan.GetStringAsync($"/api/incidents/{DbSeeder.AliceIncidentId}");

        Assert.Equal(HttpStatusCode.OK, internalComment.StatusCode);
        Assert.Contains("Внутрішня примітка", details, StringComparison.Ordinal);
    }
}
```

```
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release --filter "FullyQualifiedName
~A12_"
Восстановление завершено (1,0 с)
  SecureLab.Api net10.0 успешно выполнено (0,8 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,5 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (7,2 с)

Сводка теста: всего: 2; сбой: 0; успешно: 2; пропущено: 0; длительность: 7,2 с
Сборка успешно выполнено через 10,8 с
```

Рисунок 16 — A-12: доступ до дочірнього ресурсу

Аудит. Для відмовленої привілейованої дії записано structured audit event authorization.denied із server-verified subject id, action, resource, result, server-generated UTC-часом і тим самим request trace id, що й у безпечній відповіді клієнтові. У подію не потрапляють cookie, password, token, повний request body чи stack trace (A-09).

```
PS D:\laba1> $s = New-Object Microsoft.PowerShell.Commands.WebRequestSession
PS D:\laba1> $login = @{ userName = 'alice'; password = $env:SeedUsers__Password } | ConvertTo-Json
PS D:\laba1> Invoke-WebRequest -UseBasicParsing -Method Post http://localhost:5080/api/auth/login -ContentType 'appli
cation/json' -Body $login -WebSession $s | Out-Null
PS D:\laba1> try { Invoke-WebRequest -UseBasicParsing -Method Post "http://localhost:5080/api/incidents/20000000-0000
-0000-0000-000000000003/triage" -WebSession $s } catch { $r = $_.Exception.Response; [int]$r.StatusCode; (New-Object
IO.StreamReader($r.GetResponseStream())).ReadToEnd() }
403
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.4","title":"Заборонено","status":403,"detail":"Дію не дозво
лено для вашої ролі.","traceId":"00-3c9b74b8c98676e0a6703bc674943293-a4fb4c49ace0fd78-00"}
warn: Program[0]
      authorization.denied subject=10000000-0000-0000-0000-000000000001 action=incident.triage resource=20000000-0000-00
00-0000-000000000003 result=denied occurredAtUtc=2026-10-20T21:48:28.0469259+00:00 trace=00-3c9b74b8c98676e0a6703bc67494
3293-a4fb4c49ace0fd78-00
```

Рисунок 17 — A-09: безпечна аудиторська подія

Контрольна точка CP-02. Показано заборонений і дозволений transition, визначення наступного статусу сервером, контекст New → Triaged, доступ Morgan до коментаря через parent і недоступність internal comment для Reporter, а також збіг trace id у відповіді й події аудиту.

6. ВІДМІННИЙ РІВЕНЬ

До наявних search і severity-summary застосовано те саме правило видимості як обмеження запиту. Для Alice severity-summary після обмеження рахує лише власні інциденти (Low 2, Medium 1, High 0, Critical 0), а search?q=O’Brien повертає порожній масив, бо відповідний інцидент належить Bob. Чужі записи й агреговані дані не розкриваються (A-11); Analyst, навпаки, бачить усі дані для аналізу.

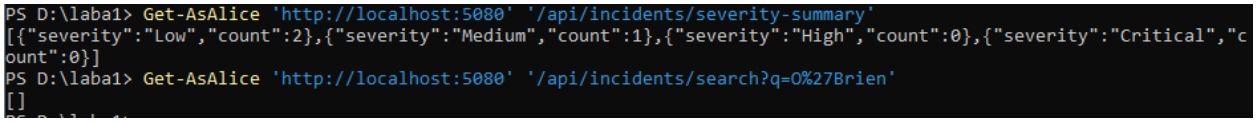


Рисунок 18 — A-11: власні endpoint-и ЛР 1–2 обмежено

Додаткова проблема. Окрім details, до виправлення ті самі два маршрути ЛР 1–2 не враховували власника. Severity-summary рахував усі інциденти таблиці, тож Alice бачила агреговані дані Bob (Low 3, Medium 1, High 1 замість власних Low 2, Medium 1), а пошук search?q=O’Brien повертав інцидент Bob (id …0004, «O’Brien classroom report»); крім того, маршрут пошуку не мав вимоги автентифікації. Це другий шлях горизонтального витоку, який не видно за перевіркою лише details. Виправлення — обмеження за суб’єктом у самому запиті (до групування та до сортування й Take(50)) і RequireAuthorization на пошуку. Стан «до» знято на vulnerable commit у окремій копії робочого дерева (git worktree, порт 5081) на тих самих даних після reset, стан «після» — на fixed коді (порт 5080); копію видалено, робоча гілка не змінювалась. Стан «після» показано на Рисунку 18, стан «до» — на Рисунку 19, а автоматично їх закріплюють тести A11_Reporter_SearchAndSummary_AreLimitedToOwnData і A11_Analyst_SearchAndSummary_SeeAllIncidents.



Рисунок 19 — Відмінний рівень: додаткова проблема й виправлення

Обґрунтування least privilege і видимості. Кожна роль має лише права матриці: Reporter — власні ресурси (видалення лише поки статус New), Analyst — читання всіх інцидентів для аналізу й один дозволений перехід, Administrator — лише власні ресурси, бо додаткових прав предметне правило не публікує. Політика 404 обрана для чужого ресурсу, щоб не розкривати його існування; 403 застосовано там, де користувач уже знає про ресурс або дію закрито роллю. Охоплено read, list, update, delete, triage, child, search і severity-summary. Залишковий ризик: CSRF для cookie-mutation до ЛР 6 (задокументований навчальний борг); конкурентні виклики triage одного інциденту не захищено окремим concurrency token (сервер читає статус перед зміною, але гонка двох одночасних запитів не перевірялась).

7. BROWSER FLOW

Після входу як Alice на auth-lab.html і переходу на головну сторінку деталі власного інциденту Alice відображаються через textContent / createTextElement(...), а не innerHTML. Запит деталей ресурсу Bob (клік у списку або готовий probe-bob-details.js у DevTools Console) дає узгоджене з політикою повідомлення відмови «Помилка: Інцидент не знайдено», однакове для чужого й неіснуючого id. Клієнт лише показує рішення сервера; приховування елемента чи client-side умова не замінюють server-side authorization. На знімках немає значень credentials, cookie чи повного Set-Cookie. Список на головній сторінці показує лише три власні інциденти Alice, а виклик loadIncidentDetails для id Bob, набраний вручну в Console, дає в консолі браузера лише 404 від сервера.

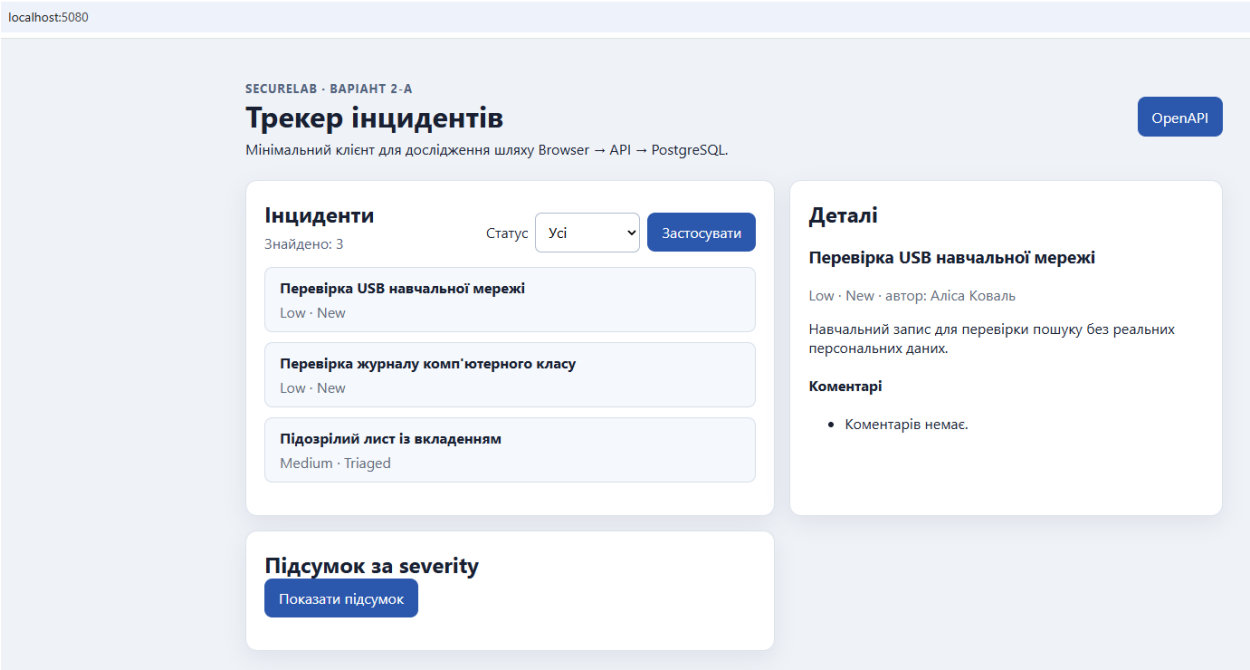


Рисунок 20 — Browser: дозволений результат Alice

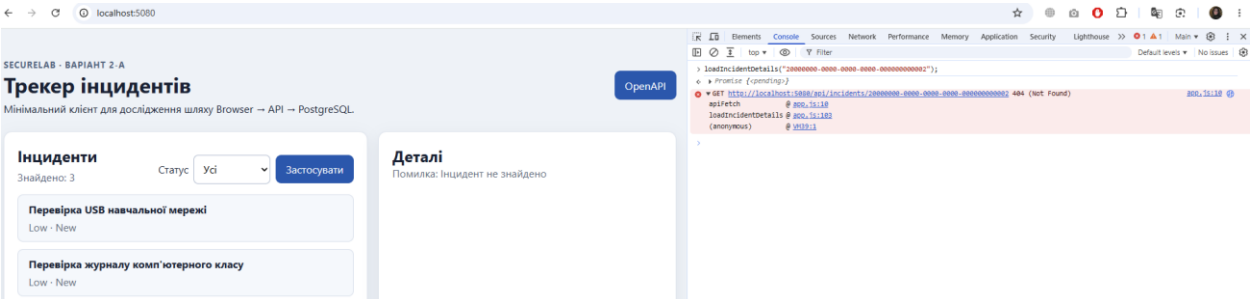


Рисунок 21 — Browser: узгоджена відмова для Bob

8. ПЕРЕВІРКИ, ВІДТВОРЕННЯ І ПОДАННЯ

Після останньої зміни коду відновлено локальний стан і запущено всі тести. Автоматизовано: A-01, A-02, A-03, A-03R, A-04, A-05, A-06, A-07, A-08a, A-10, A-11 і A-12; A-09 (audit) підтверджено ручною перевіркою зі збігом trace id. Для кожної негативної mutation-перевірки стан ресурсу читається повторно й порівнюється до/після. Пароль тестів читається зі змінної середовища й не друкується у failure message. Фактичний результат: 40 тестів, збоїв 0.

```
docker compose --env-file infra/.env.example -f infra/compose.yaml up -d --wait
dotnet run --project src/SecureLab.Api -- --reset-database
dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj --configuration
Release
```

```
PS D:\laba1> dotnet run --project src/SecureLab.Api -- --reset-database
Используются параметры запуска из src\SecureLab.Api\Properties\launchSettings.json...
Сборка…
Відновлення початкових локальних навчальних даних...
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (35ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
Локальні навчальні дані очищено та повторно заповнено seed-значеннями.
PS D:\laba1> dotnet build tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release
Восстановление завершено (1,0 с)
  SecureLab.Api net10.0 успешно выполнено (0,7 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,5 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll

Сборка успешно выполнено через 3,3 с
PS D:\laba1> dotnet test tests/SecureLab.Api.Tests/SecureLab.Api.Tests.csproj -c Release
Восстановление завершено (0,9 с)
  SecureLab.Api net10.0 успешно выполнено (0,7 с) → src\SecureLab.Api\bin\Release\net10.0\SecureLab.Api.dll
  SecureLab.Api.Tests net10.0 успешно выполнено (0,4 с) → tests\SecureLab.Api.Tests\bin\Release\net10.0\SecureLab.Api
.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.13)
[xUnit.net 00:00:29.34]   Finished:    SecureLab.Api.Tests
  SecureLab.Api.Tests (тест) net10.0 успешно выполнено (30,8 с)

Сводка теста: всего: 40; сбой: 0; успешно: 40; пропущено: 0; длительность: 30,7 с
Сборка успешно выполнено через 33,9 с
```

Рисунок 22 — Результат автоматизованих тестів

Перевірений fixed стан позначено анотованим тегом v0.4.0; тег вказує на перевірений fixed commit і не пересувався. Гілку й тег передано до приватного remote; vulnerable commit лишається лише в приватному репозиторії. Fixed commit: e56926850bb1913db1f6cad4085efd47093f954d; на ньому стоїть тег v0.4.0, graph показує його над vulnerable commit.

```
git status
git tag -a v0.4.0 -m "Laboratory work 4 completed"
git show --no-patch v0.4.0
git rev-parse HEAD
git push
```

```
git push origin v0.4.0
git log --oneline --graph -6
```

```
PS D:\laba1> git commit -m "Authorize incident access by owner and role"
[lab/4-authorization e569268] Authorize incident access by owner and role
 13 files changed, 729 insertions(+), 37 deletions(-)
 create mode 100644 src/SecureLab.Api/Application/Access/IncidentAccess.cs
 create mode 100644 src/SecureLab.Api/Presentation/Contracts/IncidentRequests.cs
 create mode 100644 src/SecureLab.Api/Presentation/Endpoints/IncidentMutationEndpoints.cs
 create mode 100644 tests/SecureLab.Api.Tests/Lab04MutationTests.cs
 create mode 100644 tests/SecureLab.Api.Tests/Lab04ReadAccessTests.cs
PS D:\laba1> git rev-parse HEAD
e56926850bb1913db1f6cad4085efd47093f954d
PS D:\laba1> git push
Enumerating objects: 47, done.
Counting objects: 100% (47/47), done.
Delta compression using up to 12 threads
Compressing objects: 100% (24/24), done.
Writing objects: 100% (27/27), 12.69 KiB | 928.00 KiB/s, done.
Total 27 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 11 local objects.
To https://github.com/vkotenok35-hub/securelab-kotenok.git
   8396023..e569268  lab/4-authorization -> lab/4-authorization
PS D:\laba1> git status --short
PS D:\laba1> git tag -a v0.4.0 -m "Laboratory work 4 completed"
PS D:\laba1> git show --no-patch v0.4.0
tag v0.4.0
Tagger: Вікторія Котенок <vkotenok35@gmail.com>
Date:   Wed Oct 21 18:05:24 2026 +0300

Laboratory work 4 completed

commit e56926850bb1913db1f6cad4085efd47093f954d (HEAD -> lab/4-authorization, tag: v0.4.0, origin/lab/4-authorization
)
Author: Вікторія Котенок <vkotenok35@gmail.com>
Date:   Wed Oct 21 18:04:09 2026 +0300

    Authorize incident access by owner and role
PS D:\laba1> git push origin v0.4.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 207 bytes | 69.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/vkotenok35-hub/securelab-kotenok.git
 * [new tag]         v0.4.0 -> v0.4.0
```

Рисунок 23 — Тег v0.4.0 на перевіреному fixed commit

```
PS D:\laba1> git log --oneline --graph -6
* e569268 (HEAD -> lab/4-authorization, tag: v0.4.0, origin/lab/4-authorization) Authorize incident access by owner a
nd role
* 8396023 Reproduce IDOR on incident details
* 95c7d9e (tag: v0.3.0, origin/lab/3-authentication, lab/3-authentication) Implement cookie authentication and protec
ted incident creation
* 4275dbe Install lab-03-start-v1 scaffold
* f63e6d0 (tag: v0.2.0, origin/lab/2-input-sqli, lab/2-input-sqli) Add LR2 report and evidence
* 7d643df Validate incident input and secure search query
```

Рисунок 24 — Історія комітів ЛР 4

9. ОСНОВНИЙ FINDING І ЗАЛИШКОВИЙ РИЗИК

Контекст → гіпотеза. Після автентифікації ЛР 3 будь-який запит із валідною cookie міг звернутися до details будь-якого інциденту за id. Гіпотеза: маршрут перевіряє автентифікацію, але не право на конкретний ресурс.

Стан до й hash. Vulnerable commit 8396023d73308eddb4b31acef2993853514925a4: Alice із власною cookie читає ресурс Bob (200, дані Bob); додатково search і severity-summary не обмежені за власником.

Першопричина. Пошук лише за id без порівняння власника з principal і без ресурсного рішення.

Fix і DTO. Fixed commit e56926850bb1913db1f6cad4085efd47093f954d: відокремлений resource-based handler для окремого ресурсу, обмеження запиту за owner для list, search і severity-summary, узгоджена 404; request DTO не дозволяє підмінити OwnerUserId, role чи status.

Retest і позитивна регресія. Alice → Bob дає 404 без даних Bob (однаково з неіснуючим id); Alice → власний ресурс лишається 200; привілейована дія закрита від Reporter і дозволена Analyst за контекстом.

Залишковий ризик. CSRF для cookie-mutation вводиться в ЛР 6 (навчальний борг); захист від гонки двох одночасних triage не додавався; стенд локальний і не є промисловою системою.

ДОКАЗОВА ТАБЛИЦЯ

Рядки заповнено за обраним відмінним рівнем (усі нижчі включено). «Фактично» — власний результат, а не копія очікування.

Таблиця 3 — Доказова таблиця A-01…A-12

ID	Рівень	Сценарій	Передумови	Дія	Очікувано	Фактично	Доказ
A-01	добрий	Анонімний доступ	немає authentication context	protected request	401, не HTML redirect	401, application/problem+json, без Location; не HTML redirect	тест A01_AnonymousDetails_Returns401WithoutRedirect; рис. 22
A-02	достатній	Власний ресурс	Alice має власний seed resource	GET /api/incidents/{Alice id}	allow	200; id …0001, власник «Аліса Коваль»	тест A02_Owner_ReadsOwnIncident; рис. 9
A-03	достатній	Чужий ресурс	Alice, ресурс Bob	GET із підміненим id	404; даних Bob немає	404 «Інцидент не знайдено»; ні назви, ні власника Bob у відповіді немає	тест A03_OtherReporter_GetsNotFound_WithoutForeignData; рис. 10
A-03R	достатній	Аналітичний read	Morgan (Analyst), ресурс Alice	GET /api/incidents/{Alice id}	allow для аналізу	200 для Morgan (Analyst), id …0001	тест A03R_Analyst_ReadsReporterIncidentForAnalysis; рис. 12
A-04	добрий	Чужий ресурс, mutation	Alice, ресурс Bob	update/delete з підміненим id	deny, стан без змін	update і delete чужого ресурсу — 404; знімок стану Bob (власник, вміст, severity,	тест A04_OtherReporter_UpdateAndDelete_AreDenied_AndDoNotChangeIncident; рис. 22

ID	Рівень	Сценарій	Передумови	Дія	Очікувано	Фактично	Доказ
						status, UpdatedAt) без змін	
A-05	добрий	List	Alice	list	лише власні записи	у списку Alice є …0001, …0003, …0005, немає ресурсів Bob; у браузері 3 інциденти	тест A05_List_ContainsOnlyOwnIncidents; рис. 13, 20
A-06	добрий	Вертикальне підвищення	звичайний Reporter	privileged transition	deny, status без змін	403; status власного інциденту New не змінився	тест A06_Reporter_Triage_Returns403_AndStatusUnchanged; рис. 14
A-07	добрий	Дозволена привілейована дія	Analyst, status New	triage New→Triaged	allow	200 і status Triaged; повторний triage — 409	тест A07_Analyst_Triage_MovesNewToTriaged_AndRepeatConflicts; рис. 14
A-08a	добрий	Overposting (ігнорується)	DTO без protected field	forged owner/role/status	content може змінитися, protected — ні	204; title змінився, owner (Alice) і status (New) без змін, id не підмінено	тест A08a_UpdateWithForgedProtectedFields_ChangesContentOnly; рис. 15
A-09	добрий	Audit event	denied privileged request	знайти authorization.denied	подія без секретів, той самий trace	403 на triage від Alice; подія authorization.denied з subject …0001, action incident.triage, resource …0003, result denied, UTC; trace у події	рис. 17

ID	Рівень	Сценарій	Передумови	Дія	Очікувано	Фактично	Доказ
						збігається з traceId у відповіді	
A-10	добрий (404)	Нерозрізнюваність 404	той самий client; чужий і невідомий id	два однакові запити	однакові status, Content-Type і тіло без traceId	обидві відповіді 404 application/problem+json; тіла без traceId збігаються (JsonNode.DeepEquals)	тест A10_ForeignAndUnknownId_ReturnIdenticalNotFound; рис. 11
A-11	відмінний	Власні endpoint-и ЛР 1–2	Alice; search і severity-summary	виконати обидва	лише дозволена множина	Alice: Low 2, Medium 1, High 0; search O'Brien — []. До виправлення (vulnerable): Low 3, Medium 1, High 1; search повертав …0004	тести A11_Reporter_…, A11_Analyst_…; рис. 18, 19
A-12	добрий	Child resource	Alice (Reporter), comment Morgan→ Bob	GET /api/incident-comments/{id}	deny; internal не в проєкції Reporter	чужий і internal коментарі — 404; власний звичайний — 200; internal-текст відсутній у details Reporter; Analyst читає internal (200)	тести A12_Reporter_CannotReadForeignOrInternalComment, A12_Analyst_ReadsInternalComment_ThroughParent; рис. 16

ВИКОРИСТАННЯ ШІ

Використання: так, виключно як допоміжний інструмент для структурування звіту та швидкої навігації по документації; використано Claude (Anthropic).

ШІ залучався виключно для:

- Допомоги у формуванні та вичитці розділів звіту за встановленим шаблоном.
- Консультацій щодо синтаксису окремих методів ASP.NET Core та CLI-команд Git.

Проєктування матриці доступу, написання коду захисту, реалізація тестів A-01…A-12, відтворення PoC IDOR, налаштування середовища та повна верифікація результатів виконані повністю самостійно.

Таблиця 4 — Журнал використання ШІ

Дата / інструмент	Для чого	Що використано	Що перевірено самостійно
20.10.2026 / Claude	Розбір методички ЛР 4, каркас звіту	Каркас розділів за шаблоном методички	План виконання роботи, вибір архітектури та матриці доступу
20.10.2026 / Claude	Довідка з синтаксису	Уточнення сигнатур методів і команд Git	Написання коду ендпоінтів, тестів (40/40), збірка, PoC і фіксація тегу v0.4.0

ВИСНОВОК

У ЛР 4 авторизацію відокремлено від автентифікації: сервер для кожного запиту вирішує «суб’єкт + дія + ресурс + контекст», а не довіряє самому факту cookie чи переданому id. Горизонтальне підвищення привілеїв (IDOR/BOLA) на головній action details відтворено на vulnerable commit і усунуто відокремленим resource-based handler; чужий ресурс повертає 404, однаково з неіснуючим, а власний лишається доступним.

Для доброго й відмінного рівнів list обмежено дозволеною множиною, update/delete і дочірній ресурс дістали узгоджену перевірку, вертикальне підвищення закрито рольовим рішенням із контекстом стану, overposting не змінює серверних полів, а відмови привілейованих дій пишуться в безпечний аудит. search і severity-summary обмежено за суб’єктом (A-11), усунуто додатковий витік через агреговані дані та пошук. Поведінку підтверджено сценаріями A-01…A-12 (40 тестів, збоїв 0), а перевірений fixed commit e569268 зафіксовано тегом v0.4.0 над vulnerable commit 8396023.